

Musterlösungen

01 | a) `true`, `false`

Der Datentyp `boolean` kann nur zwei Werte haben: `true` und `false`.

02 | b) `boolean b = true;`

Ein `boolean`-Wert wird mit den Werten `true` oder `false` definiert, z. B. `boolean b = true;`.

03 | b) `true` wird zu 1 und `false` wird zu 0.

In Java ist es nicht möglich, direkt einen `boolean` in einen `int` zu konvertieren. Aber bei explizitem Casting würde `true` als 1 und `false` als 0 interpretiert.

04 | b) `3 == 3`

Der Ausdruck `3 == 3` gibt `true` zurück, weil die beiden Werte gleich sind.

05 | a) `true` wenn beide Operanden `true` sind, sonst `false`

Der `&&`-Operator (logisches UND) gibt `true` zurück, wenn beide Operanden `true` sind. Ansonsten gibt er `false` zurück.

06 | b) Der Block im `if` wird übersprungen.

In einer `if`-Bedingung wird der Block nur ausgeführt, wenn der `boolean`-Wert `true` ist. Ist er `false`, wird der Block übersprungen.

07 | b) `true`

Die Methode `Boolean.parseBoolean("true")` gibt den Wert `true` zurück, wenn der String „true“ übergeben wird. Andere Werte führen zu `false`.

08 | a) `x == y`

Der Ausdruck `x == y` vergleicht zwei Werte und gibt einen `boolean`-Wert zurück, je nachdem, ob die Werte gleich sind.

09 | c) Negation

Der Operator `!` ist der logische „Nicht“-Operator und wird verwendet, um den Wert eines `boolean`-Werts zu negieren.

10 | b) Die Methode erhält den Wert und kann ihn weiterverarbeiten.

Wenn ein `boolean`-Wert als Parameter in einer Methode übergeben wird, erhält die Methode den Wert und kann ihn im Methodenkörper verwenden.
